

Arquitetura de Computadores

Eng. Informática, ESTiG/IPB

2023/2024

José Rufino

(baseado no Capítulo 11 do livro “The Essentials of Computer Organization and Architecture” de Linda Null e Julia Lobur)

Unidade 7: Medição e Otimização de Desempenho



THE ESSENTIALS OF
**Computer
Organization and
Architecture**

FIFTH EDITION

**Linda Null
Julia Lobur**

Objetivos

- entender as formas de medição do desempenho de computadores
- ser capaz de descrever os métodos mais vulgares de medição e comparação de desempenho e perceber as suas limitações
- ficar familiarizado com os fatores que contribuem para melhorar o desempenho da CPU
- saber avaliar o impacto de cada componente no desempenho global do sistema de computação

7.1 Introdução



- as ideias aqui apresentadas ajudarão na compreensão de variadas medições do desempenho de computadores
- os conceitos de base poderão ser aplicados durante a compra de um sistema de grande porte ou atualização de um sistema existente
- serão discutidos fatores com influência no desempenho de um sistema, incluindo aspectos que permitem melhorar o desempenho de programas

7.2 Ferramentas Matemáticas



- o foco da avaliação do desempenho depende dos aspetos do mesmo considerados mais relevantes
- um **utilizador** estará mais preocupado com o **tempo de resposta** (reatividade) e **de retorno** (quanto tempo é necessário para o sistema terminar uma dada tarefa)
 - importa minimizar o mais possível estes tempos
- um **administrador** estará mais preocupado com o **débito** (quantas tarefas são concluídas por cada u.t.)
 - importa maximizar o mais possível este débito
- a otimização do desempenho deve procurar responder em simultâneo, a estes dois requisitos “contraditórios”

7.2 Ferramentas Matemáticas

- Speedup



- para comparar o desempenho de sistemas com base numa tarefa, mede-se o tempo que demoram a efetuar essa tarefa
- sendo $T(A)$ e $T(B)$ o tempo de execução da mesma tarefa nos sistemas A e B, o desempenho relativo dos sistemas é:

$$S(A,B) = T(B)/T(A)$$

= **speedup** de A em relação a B

$$S(B,A) = T(A)/T(B)$$

= **speedup** de B em relação a A

-

$$[S(X,Y) - 1] \times 100 = m\%$$

- se $S(X,Y) > 1$, então $m > 0$ e X é **m%** mais rápido que Y
- se $S(X,Y) < 1$, então $m < 0$ e X é **|m%|** mais lento que Y (ou, equivalentemente, X tem $S(X,Y)$ do desempenho de Y)

7.2 Ferramentas Matemáticas

- Speedup



- exemplo:

checkpoint: exercício FM1

- um sistema A executa um programa em 3 minutos, e um sistema B executa esse mesmo programa em 4 minutos
- a aceleração do sistema A em relação ao sistema B é **1.33** (>1) pelo que A é **33% mais rápido** que B

$$S(A,B) = \frac{T(B)}{T(A)} = \frac{4}{3} = \mathbf{1.33}$$

$$(\mathbf{1.33} - 1) \times 100 = \mathbf{33\%}$$

- a aceleração do sistema B em relação ao sistema A é **0.75** (<1) pelo que B é **25% mais lento que A** (ou, equivalentemente, B apresenta **75% do desempenho de A**)

$$S(B,A) = \frac{T(A)}{T(B)} = \frac{3}{4} = \mathbf{0.75}$$

$$(\mathbf{0.75} - 1) \times 100 = \mathbf{-25\%}$$

7.2 Ferramentas Matemáticas

- Média Aritmética



- na avaliação do desempenho de um sistema, é habitual executar um **conjunto representativo de várias tarefas**
- esse conjunto define uma certa **carga de trabalho**
- para cada tarefa haverá uma **medida de desempenho**
- essas medidas podem agregar-se recorrendo a **médias**
- a média mais conhecida é a **média aritmética**, dada por:

$$\frac{\sum_{i=1}^n x_i}{n}$$

7.2 Ferramentas Matemáticas

- Média Aritmética

- mas a média aritmética pode induzir em erro, como acontece com dados muito dispersos / enviesados
 - considerem-se os tempos de execução da tabela abaixo; as diferenças de desempenho são escondidas pela simples média
 - **os 3 sistemas parecem ter igual desempenho, dado que as três médias aritméticas têm exatamente o mesmo valor !**

Program	System A Execution Time	System B Execution Time	System C Execution Time
v	50	100	500
w	200	400	600
x	250	500	500
y	400	800	800
z	5000	4100	3500
Average	1180	1180	1180

7.2 Ferramentas Matemáticas

- Média Aritmética Pesada



- se as frequências de execução (i.e., as cargas de trabalho esperadas) forem conhecidas, será mais adequado recorrer a uma **média (aritmética) pesada**
 - para as frequências abaixo, a média pesada do sistema A é
$$50 \times 0.5 + 200 \times 0.3 + 250 \times 0.1 + 400 \times 0.05 + 5000 \times 0.05 = 380,$$
e para o sistema C é $500 \times 0.5 + 600 \times 0.3 + \dots + 3500 \times 0.05 = 695$

Program	Execution Frequency	System A Execution Time	System C Execution Time
v	50%	50	500
w	30%	200	600
x	10%	250	500
y	5%	400	800
z	5%	5000	3500
Weighted Average		380 seconds	695 seconds

- agora pode-se dizer que o sistema A é **83%** mais rápido que o sistema C para esta carga em particular ($695 / 380 = 1,83$)

7.2 Ferramentas Matemáticas

- Média Aritmética Pesada

checkpoint:
exercício FM2

- no entanto, é necessário levar em conta que as cargas de trabalho podem variar ao longo do tempo
 - um sistema otimizado para uma dada carga de trabalho pode ter um um pior desempenho quando a carga de trabalho muda
 - e por isso o ranking dos sistemas pode até inverte-se !

Program	Execution Frequency	System A Execution Time	System C Execution Time
v	25%	50	500
w	5%	200	600
x	10%	250	500
y	5%	400	800
z	55%	5000	3500
Weighted Average		2817.5 seconds	2170 seconds

- agora pode-se dizer que o sistema C é **30%** mais rápido que o sistema A para esta carga em particular ($2817,5 / 2170 \approx 1,30$)

7.2 Ferramentas Matemáticas

- Média Geométrica



- a média aritmética é inadequada com dados espalhados/enviesados
- a média pesada é adequada para cargas representativas e estáticas
- a **média geométrica** fornece um número consistente com o qual se podem fazer comparações, independente/ da distribuição dos dados
 - vs média aritmética: a geométrica dá mais importância às medições (x_n) pequenas e é menos afetada por flutuações das medições
 - o seu valor é a “**raíz de índice n do produto das n medições**”

$$G = \left[x_1 \cdot x_2 \cdot x_3 \cdot \dots \cdot x_n \right]^{\frac{1}{n}}$$

7.2 Ferramentas Matemáticas

- Média Geométrica

- é habitual utilizar a média geométrica após **normalizar** os tempos de execução com base num **sistema de referência**
 - abaixo, os tempos são normalizados com base no sistema B

Program	System A Execution Time	Execution Time Normalized to B	System B Execution Time	Execution Time Normalized to B	System C Execution Time	Execution Time Normalized to B
v	50	2	100	1	500	0.2
w	200	2	400	1	600	0.6667
x	250	2	500	1	500	1
y	400	2	800	1	800	1
z	5000	0.82	4100	1	3500	1.1714
Geometric Mean		1.6733		1		0.6898

$$\frac{\text{System B Execution Time}}{\text{System A Execution Time}}$$

$$\frac{\text{System B Execution Time}}{\text{System C Execution Time}}$$

7.2 Ferramentas Matemáticas

- Média Geométrica

- como é de esperar, quando é usado um outro sistema para sistema de referência, obtêm-se valores diferentes
 - abaixo, os tempos são normalizados com base no **sistema C**

Program	System A Execution Time	Execution Time Normalized to C	System B Execution Time	Execution Time Normalized to C	System C Execution Time	Execution Time Normalized to C
v	50	10	100	5	500	1
w	200	3	400	1.5	600	1
x	250	2	500	1	500	1
y	400	2	800	1	800	1
z	5000	0.7	4100	0.8537	3500	1
Geometric Mean		2.4258		1.4497		1

$$\frac{\text{System C Execution Time}}{\text{System A Execution Time}}$$

$$\frac{\text{System C Execution Time}}{\text{System B Execution Time}}$$

7.2 Ferramentas Matemáticas

- Média Geométrica



- mas, seja qual for o sistema de referência escolhido, **a razão das médias geométricas é consistente**
 - isto pode-se confirmar abaixo, quando se usa o sistema B ou o sistema C como sistema de referência

System A Execution Time	Execution Time Normalized to B	System B Execution Time	Execution Time Normalized to B	System C Execution Time	Execution Time Normalized to B
Geometric Mean	1.6733		1		0.6898

$$1,6733 / 1 = 2,4258 / 1,4497$$

System A Execution Time	Execution Time Normalized to C	System B Execution Time	Execution Time Normalized to C	System C Execution Time	Execution Time Normalized to C
Geometric Mean	2.4258		1.4497		1

$$1 / 0,6898 = 1,4497 / 1$$

7.2 Ferramentas Matemáticas

- Média Geométrica



- Usando a média geométrica, ou seus rácios, podem-se ordenar (ranking) sistemas com base no seu desempenho
 - para o exemplo anterior, pode-se dizer que o sistema A é o mais rápido, seguido do B e este do C – comparação relativa
- Mas médias geométricas, ou seus rácios, não são *speedups*
 - para o exemplo anterior, embora o rácio das médias geométricas do sistema C e do sistema A seja ≈ 2.43 , isso não implica que o sistema A é ≈ 2.43 vezes mais rápido que o C
- Ao contrário das médias aritméticas, a média geométrica não traduz um valor absoluto de desempenho do sistema; é apenas uma ferramenta de comparação, para os seriar.

7.2 Ferramentas Matemáticas - Média Geométrica



- problema inerente à utilização da média geométrica:
 - contribuição equivalente de todos os tempos para o resultado
- logo, a redução do tempo de execução de um pequeno programa em 10% terá o mesmo impacto que a redução em 10% do tempo de execução de um grande programa
 - pequenos programas são, geralmente, mais fáceis de otimizar, mas, numa situação real, pretender-se-á reduzir o tempo de execução dos programas mais extensos !

7.2 Ferramentas Matemáticas

- Média Geométrica



Program	System A Execution Time	Execution Time Normalized to C	System B Execution Time	Execution Time Normalized to C	System C Execution Time	Execution Time Normalized to C
v	45	10	90	5	450	1
w	200	3	400	1,5	600	1
x	250	2	500	1	500	1
y	400	2	800	1	800	1
z	5000	0,7	4100	0,8537	3500	1

Geometric
Mean

2,4258

1,4497

1,0000

Program	System A Execution Time	Execution Time Normalized to C	System B Execution Time	Execution Time Normalized to C	System C Execution Time	Execution Time Normalized to C
v	50	10	100	5	500	1
w	200	3	400	1,5	600	1
x	250	2	500	1	500	1
y	400	2	800	1	800	1
z	4500	0,7	3690	0,8537	3150	1

Geometric
Mean

2,4258

1,4497

1,0000

7.2 Ferramentas Matemáticas

- Média Geométrica Pesada



- a **média geométrica** tem também uma variante **pesada**
 - cada termo x_i é elevado a um peso específico w_i
 - assumindo que $w_1 + w_2 + w_3 + \dots + w_n = 1$ então

$$G = (x_1^{w_1} * x_2^{w_2} * x_3^{w_3} * \dots * x_n^{w_n})$$

- formulação adequada quando a importância de cada termo x_i (oriundo de um sub-benchmark diferente) é diferente
- para $w_i = 1/n$, obtém-se a expressão da média não pesada
- ver também <https://dl.acm.org/doi/pdf/10.1145/5666.5673>

checkpoint: exercício FM3

7.2 Ferramentas Matemáticas - Média Harmónica



- nem a média aritmética nem a geométrica são adequadas para valores (x_n) expressos como **taxas**
 - taxa (ou débito) = número de operações por segundo
- a **média harmónica** permite formular uma estimativa matemática do débito e permite comparar os débitos relativos de sistemas e componentes de sistemas
- para saber a média harmónica, divide-se o nº total de taxas pelo somatório dos inversos de todas as taxas

$$H = n \div (1/x_1 + 1/x_2 + 1/x_3 + \dots + 1/x_n)$$

7.2 Ferramentas Matemáticas - Média Harmónica



- exemplo:
 - um programa A com 3000 instruções executou à taxa de 3000 instr./us durante as primeiras **1000** instruções, à taxa de 4000 instr./us durante as segundas **1000** instruções, e à taxa de 6000 instr./us durante as últimas **1000** instruções
 - média aritmética das taxas: **4333.3(3)** instr./us (errado !!)
 - média geométrica das taxas: **4160.17** instr./us (errado !!)
 - tempo de execução das 1as 1000 instruções: $1000/3000 = 1/3$ us
 - tempo de execução das 2as 1000 instruções: $1000/4000 = 1/4$ us
 - tempo de execução das 3as 1000 instruções: $1000/6000 = 1/6$ us
 - tempo total de execução: $\frac{3}{4}$ us; **tx. média de execução** = 3000 instr. / $\frac{3}{4}$ us = **4000** instr./us ; **média harmónica** = $3 / (1/3000 + 1/4000 + 1/6000) = \mathbf{4000}$ instr./us (correto !!)

7.2 Ferramentas Matemáticas

- Média Harmónica Pesada



- a **média harmónica** tem também uma variante **pesada**
 - o inverso de cada taxa ($1/x_i$) é afetado de um peso w_i
 - assumindo que $w_1 + w_2 + w_3 + \dots + w_n = 1$ então

$$H = 1 \div (w_1/x_1 + w_2/x_2 + w_3/x_3 + \dots + w_n/x_n)$$

- esta formulação é adequada quando o número de amostras ou eventos é diferente para cada taxa x_i
 - $w_i = (\text{nº de eventos à taxa } x_i) / (\text{nº total de eventos})$
 - o exemplo seguinte ilustra uma situação deste tipo
- ver também https://en.wikipedia.org/wiki/Harmonic_mean

7.2 Ferramentas Matemáticas

- Média Harmónica Pesada

checkpoint:
exercícios
FM4A/B/C

- exemplo de média harmónica pesada:
 - um programa A com 3000 instruções executou à taxa de 3000 instr./us durante as primeiras **1500** instruções, à taxa de 4000 instr./us durante as **1000** instruções seguintes, e à taxa de 6000 instr./us durante as últimas **500** instruções
 - tem. de execução das 1^{as} 1500 instruções: $1500/3000 = 0.5$ us
 - tem. de exec. das 1000 instruções seguintes: $1000/4000 = 0.25$ us
 - tem. de exec. das últimas 500 instruções: $500/6000 = 0.083(3)$ us
 - tempo total de execução: $0.5 + 0.25 + 0.083(3) = 0.83(3)$ us; **tx. média de execução** = $3000 \text{ instr.} / 0.83(3) \text{ us} = \mathbf{3600}$ instr./us;
média harmónica pesada = $1 / [(1500/3000)/3000 + (1000/3000)/4000 + (500/3000)/6000] = \mathbf{3600}$ instr./us ; os pesos **w_i** são a proporção de instruções executadas a taxas diferentes

7.2 Ferramentas Matemáticas - Média Harmónica



- **vantagens** da média harmónica sobre a geométrica:
 - é um preditor adequado do comportamento do sistema
 - i.e., é útil além da mera comparação de desempenhos
 - taxas + baixas têm > impacto no resultado; logo, acelerando os programas + lentos, melhora-se o desempenho global do sistema (e isto é o que faz sentido na prática)
- tal como a média geométrica, a harmónica pode-se usar com rácios de desempenhos relativos, mas é mais sensível à escolha da máquina de referência ...
 - ou seja, os rácios de médias harmónicas não são tão consistentes como os de médias geométricas

7.2 Ferramentas Matemáticas

- Resumo



- a seguinte tabela resume as circunstâncias em que devem ser usadas cada uma das médias anteriores

Mean	Uniformly Distributed Data	Skewed Data	Data Expressed as a Ratio	Indicator of System Performance Under a Known Workload	Data Expressed as a Rate
Arithmetic	X			X	
Weighted Arithmetic				X	
Geometric		X	X		
Harmonic				X	X

7.2 Ferramentas Matemáticas

- Estatísticas e seus Significados

- a avaliação objetiva do desempenho de um computador é (mais) crítica no âmbito de decisões de compra
 - para sistemas de nível empresarial, este processo é complicado e as consequências de uma má decisão poderão ser graves !!
- infelizmente, as vendas de computadores dependem tanto (ou ainda mais !) do marketing associado, como do desempenho característico de cada sistema
- um comprador prudente deverá compreender como dados objetivos sobre desempenho se podem usar abusivamente, para sobrevalorizar um produto

7.2 Ferramentas Matemáticas

- Estatísticas e seus Significados

- as práticas enganosas mais comuns incluem:
 - estatísticas seletivas: apenas os resultados mais favoráveis são publicados, sendo omitidos quaisquer aspetos negativos
 - indicação de valores de desempenho máximo (*peak values*), ignorando os valores médios
 - utilização vaga das palavras “quase”, “próximo”, “mais” e “menos”, quando são feitas comparações
 - utilização de estatísticas (e.g., médias) inapropriadas
 - apelar à compra de um produto pela sua popularidade
 - adulteração de benchmarks / optimização para benchmarks

7.3 Medição e comparação de desempenho

- a medição e comparação de desempenho (ou *performance benchmarking*) é a “ciência de fazer avaliações objetivas acerca do desempenho de um dado sistema em função de outro(s) sistema(s)”; estas avaliações são também úteis para medir o efeito de atualizações
- razões do tipo preço/desempenho podem ser derivadas dos *benchmarks* vulgarmente utilizados
- o grande problema é não existir um *benchmark* de eleição que permita dizer qual o sistema capaz de executar determinadas aplicações de forma mais rápida, tendo como limite um dado orçamento

7.3 Medição e comparação de desempenho

- Velocidade da CPU

- a **velocidade da CPU** é vista, muitas vezes, como uma medida de desempenho do sistema
- de entre as formas de medição da velocidade da CPU incluem-se: ***frequência de relógio*** (MHz, GHz) e ***milhões de instruções por segundo*** (MIPS)
- porém, dizer que o sistema A é mais rápido que o sistema B porque A “corre” a 1.4GHz e B “corre” a 900MHz é válido apenas quando as arquiteturas dos conjuntos de instruções (ISAs) de A e B são idênticas
 - contudo, sistemas com ISAs diferentes podem efetivamente alcançar resultados idênticos no mesmo intervalo de tempo

7.3 Medição e comparação de desempenho

- Velocidade da CPU

- no caso da métrica **MIPS**, está em causa a taxa de execução de um mix de instruções de aritmética inteira, de vírgula flutuante, e de instruções lógicas
- no entanto, diferentes ISAs requerem um número diferente de ciclos de relógio para essas instruções, o que não é levado em conta pela métrica MIPS
- **exemplo:** um sistema CISC realiza uma divisão em 20 instruções, e um sistema RISC em 60 instruções; ambos demoram um segundo a realizar a divisão; a métrica MIPS do sistema RISC será então o triplo da métrica MIPS do sistema CISC, o que é enganador

7.3 Medição e comparação de desempenho - Velocidade da CPU

- algo similar ocorre com a métrica **MFLOPS** (*milhões de instruções de vírgula flutuante por segundo*), originalmente concebida para supercomputadores
- de facto, não há consenso sobre o que é uma FLOP
 - exemplo: para um produto realizado à custa de somas, o número destas somas é contabilizado ? se for, prejudicam-se implementações + eficientes
- apesar destas limitações, a velocidade de relógio e as métricas MIPS e MFLOPS podem ser úteis para comparar sistemas da mesma ISA ou fabricante ...

7.3 Medição e comparação de desempenho - Tempo Total de Execução

- medição em Linux

- **time .\x.exe**

- real 0m0.081s // wall-clock time (total time)

- user 0m0.072s // CPU time (x.exe is executing)

- sys 0m0.009s // non-CPU time (x.exe is blocked)

- medição em Windows

- **powershell "Measure-Command{.\x.exe | Out-Default}"**

- TotalMilliseconds : 80.3549

- ambos os casos: resolução limitada a milissegundos, fiável apenas para medir programas não muito rápidos

7.3 Medição e comparação de desempenho - Tempo Parcial de Execução

```
#include <stdio.h>
#include <sys/time.h>
int main () {
    // Start measuring time
    struct timeval begin, end;
    gettimeofday(&begin, 0);
    // Task
    int iterations = 1000*1000*1000;
    double sum = 0, add = 1;
    for (int i=0; i<iterations; i++) {
        sum += add; add /= 2.0;
    }
    // Stop measuring time
    gettimeofday(&end, 0);
    // Calculate the elapsed time
    long seconds = end.tv_sec - begin.tv_sec;
    long microseconds = end.tv_usec - begin.tv_usec;
    double elapsed = seconds + microseconds*1e-6;

    printf("Result: %.3f\n", sum);
    printf("Time: %.3f seconds.\n", elapsed);
    return 0;
}
```

- medir a **duração de partes do código**, ou de programas muito rápidos, pode-se fazer com funções da linguagem usada ou do sistemas operativo
- mais exemplos em (*)

checkpoint:
atividade TE1

7.3 Medição e comparação de desempenho

- Tempo Parcial de Execução

- outra hipótese é recorrer à instrução máquina RDTSC (Read Time-Stamp-Counter) disponível em CPUs x86
- o registo TSC conta o nº de ciclos de CPU desde o boot
- o número de ciclos pode traduzir-se facilmente em tempo, desde que a frequência da CPU se mantenha
 - **#seconds** = #cycles / frequency (Hz)
- em CPUs com núcleos com diferentes frequências (big vs LITTLE), e com frequências dinâmicas (speedstep), esta abordagem requer passos adicionais de correcção
- slide seguinte: exemplo simples de uso de RDTSC

7.3 Medição e comparação de desempenho

- Tempo Parcial de Execução

```
#include <stdio.h>
#include <stdlib.h>
#include <asm/types.h>

int main(){
    static __u32 hi1, hi2, lo1, lo2; __u64 interval;

    // Start measuring time
    asm("rdtsc; movl %%edx, %0; \
    movl %%eax, %1" : "=r" (hi1), "=r" (lo1) : : "%edx", "%eax");

    // Task
    { long a; a=random(); }
    //{ long a; scanf("%ld", &a); }

    // Stop measuring time
    asm("rdtsc; movl %%edx, %0; \
    movl %%eax, %1" : "=r" (hi2), "=r" (lo2) : : "%edx", "%eax");

    interval = (((__u64)hi2 << 32) + lo2) - (((__u64)hi1 << 32) + lo1);

    printf("Cycles: %llu\n", interval);
    printf("Time(s): %.9f\n", (double)interval/ (4*1e9));
}
```

Linux only

4GHz =
 4×10^9 Hz

7.3 Medição e comparação de desempenho

- Número de Amostras, Ambiente

- o tempo de execução deve ser medido várias vezes !
- quantas vezes ? e como lidar com as amostras ?
 - **k-best**: menor de k valores (k amostras)
 - **média k-best**: média dos k valores menores
 - **média**: média de todos os valores
 - e se o Desvio Padrão for “elevado” ?
 - **mediana**: valor central dos valores ordenados
 - **moda**: valor mais frequente
 - repetição pouco provável para tempos pequenos
 - ignorar o primeiro valor ? (e porquê ?)
- condições de benchmarking: reboot entre amostras ?
esperar que o sistema fique ocioso ? justo / realista ?

7.3 Medição e comparação de desempenho - Benchmarks Sintéticos

- com o intuito de medir e descrever desempenho independentemente da velocidade do relógio e das ISAs, têm vindo a ser criados *benchmarks sintéticos*
 - trata-se de programas destinados exclusivamente à produção de números de desempenho
 - os primeiros *benchmarks sintéticos* – *Whetstone*, *Dhrystone* e *Linpac* (entre outros) – eram relativamente pequenos e facilmente otimizáveis
 - isso limitou a sua utilidade desde o princípio
 - por isso, não servem para a avaliação de desempenho dos sistemas da atualidade

7.3 Medição e comparação de desempenho - Benchmarks Sintéticos

- em 1988, a *Standard Performance Evaluation Corporation* (SPEC) foi constituída para dar resposta à necessidade de *benchmarks* objetivos
- a SPEC produz suites de *benchmarking* para variadas classes de computadores e arquiteturas
- a maior parte dos benchmarks SPEC são pagos
- um *benchmark* SPEC é uma coleção de programas (***kernels***) que resolvem o âmago de certos problemas
 - logo, atividades acessórias do sistema que não contribuam para a resolução do problema, tal como operações de E/S em problemas CPU-bound, são ignoradas durante a avaliação

7.3 Medição e comparação de desempenho - Benchmarks Sintéticos

- o *benchmark* SPEC mais conhecido é o **SPEC CPU**
- a última versão (2017) tem 4 suites com 42 benchmarks
[<https://www.spec.org/cpu2017/Docs/index.html#benchmarks>]
- estes *benchmark* estão codificados em C, C++ e Fortran, cobrindo uma grande variedade de áreas de aplicação
- avaliam desempenho da **CPU**, **Memória** e **Compiladores**
- produzem métricas de tipo **SPECspeed** e **SPECrate**
[<https://www.spec.org/cpu2017/Docs/overview.html#Q15>]
 - SPECspeed: mede **tempo** de execução de **1 instância**
 - SPECrate: corre **várias instâncias em paralelo**; mede **throughput** (número de instâncias concluídas por u.t.)

7.3 Medição e comparação de desempenho - Benchmarks Sintéticos

- o tempo de execução de cada kernel é dividido pelo tempo de execução do mesmo kernel numa máquina de referência (Sun Fire V490 com CPU a 2100MHz)
[<https://www.spec.org/cpu2017/Docs/overview.html#Q18>]
- o resultado final do benchmark é a **média geométrica** das razões obtidas para todos os kernels executados
- os fabricantes podem então fazer referência a dois tipos de resultados: valores de **pico** e de **base**, que correspondem, respetivamente, à utilização ou não de *flags* de optimização na compilação dos kernels
[<https://www.spec.org/cpu2017/Docs/overview.html#Q16>]

7.3 Medição e comparação de desempenho - Benchmarks Sintéticos

- recorde-se que o *benchmark* SPEC CPU avalia fundamentalmente o desempenho da CPU+RAM ...
- quando o desempenho de todo o sistema, no contexto de uma elevada carga de transações, é uma preocupação, os *benchmarks* do *Transaction Performance Council* (TPC) são mais adequados
 - a atividade E/S (storage, network) é agora considerada
- a versão atual destes benchmarks é a TPC-C v5
[<https://www.tpc.org/tpcc/default5.asp>]
- a suite TPC-C modela as típicas transações de um entreposto, usando software de emulação de terminal

7.3 Medição e comparação de desempenho - Benchmarks Sintéticos

- são simuladas transações com dados gerados aleatoriamente para minimizar o impacto de caches
- 90% das transações devem completar em 5 segs.
- a métrica do TPC-C é o número de transações de encomendas por minuto (*tpmC*) que o sistema consegue processar, tenda em conta que, concorrentemente é executado um mix de outras transações
- o resultado *tpmC* é dividido pelo custo total do sistema, resultando na razão preço-desempenho
- o preço do sistema inclui todo o hardware, software e custo de manutenção que o cliente espera pagar

7.3 Medição e comparação de desempenho - Benchmarks Sintéticos

checkpoint:
atividade BS1

- o *Transaction Performance Council* também desenvolveu *benchmarks* para sistemas de suporte à decisão (usados, por exemplo, em aplicações de *data mining*) e para sistemas Web e de comércio eletrônico
[<https://www.tpc.org/information/benchmarks5.asp>]
- os resultados dos testes são auditados por uma entidade certificada e quando são publicados, os sistemas testados tem de já ser comercializados
- os *benchmarks* TPC são uma espécie de ferramenta de simulação que podem ser inclusivamente usados para otimizar o desempenho do sistema quando sujeito a condições extremas (fora do que é habitual)

7.4 Otimização do desempenho da CPU

- Técnicas de Otimização de Código

- a otimização do desempenho da CPU inclui várias técnicas já analisados em capítulos anteriores
 - unidades de vírgula flutuante integradas na CPU
 - *encadeamento de instruções* (incluindo *saltos retardados* e *previsão de saltos*, para garantir uso pleno da *pipeline*)
 - *super-pipelining* (várias pipelines) e super-escalaridade (várias unidades funcionais, como ALUs ou FPUs)
 - *caches* da CPU, múltiplos núcleos de execução (*multi-core*)
- ainda não coberto: **técnicas de otimização do código**
 - *loop unrolling / fusion / fission*, *data (in)dependence*, *loop interchange*, *function call avoidance*, etc.

7.4 Otimização do desempenho da CPU

- Técnicas de Otimização de Código

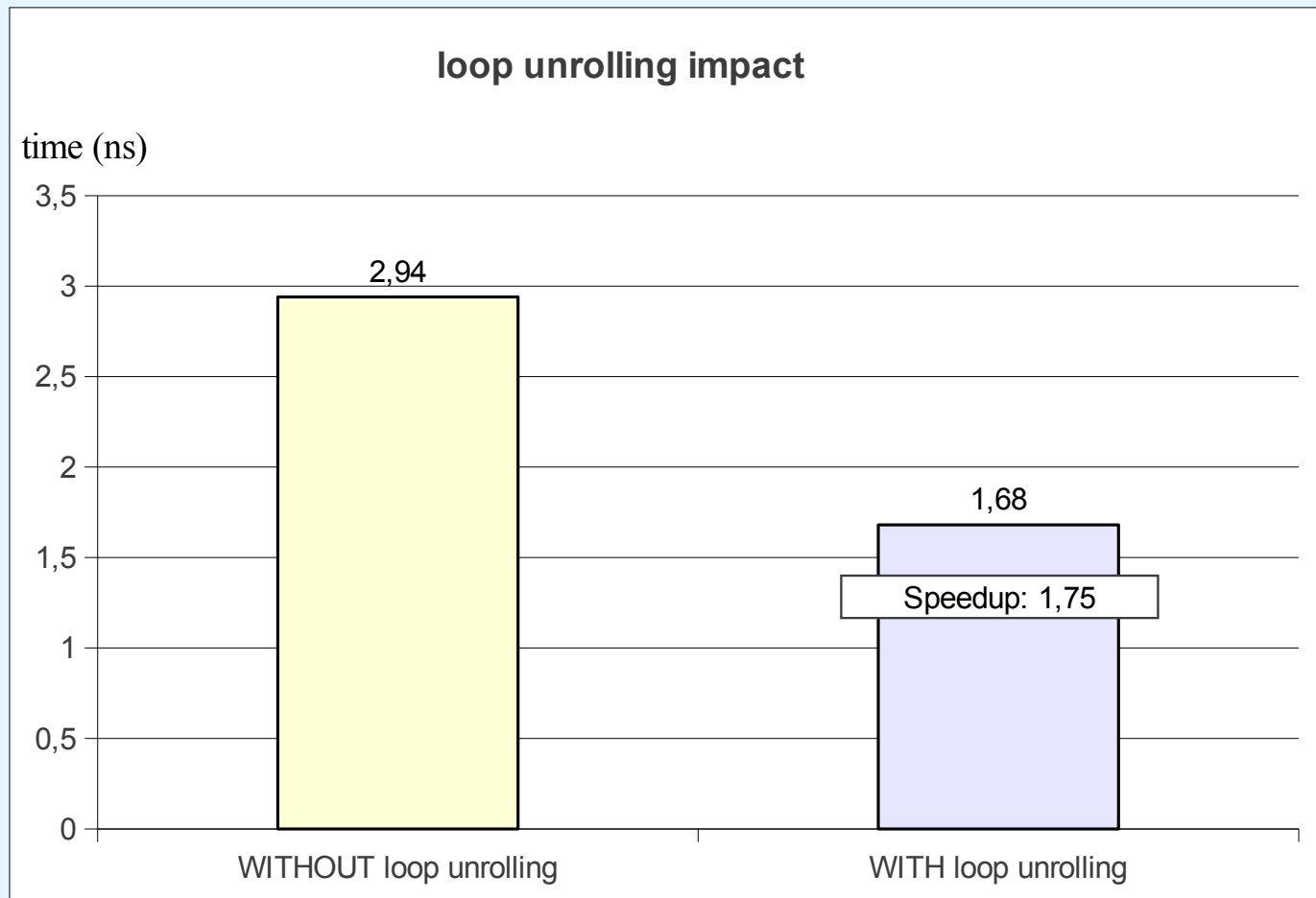
- ***loop unrolling*** (desdobramento de ciclos): maximizar o número de instruções por cada iteração de um ciclo

```
// SEM loop unrolling  
for (i=0; i<3000; i++)  
    A[i] = A[i] + B[i] * c;
```

```
// COM loop unrolling  
for (i=0; i<3000; i+=3){  
    A[i] = A[i] + B[i] * c;  
    A[i+1] = A[i+1] + B[i+1] * c;  
    A[i+2] = A[i+2] + B[i+2] * c;  
}
```


7.4 Otimização do desempenho da CPU

- Técnicas de Otimização de Código



7.4 Otimização do desempenho da CPU

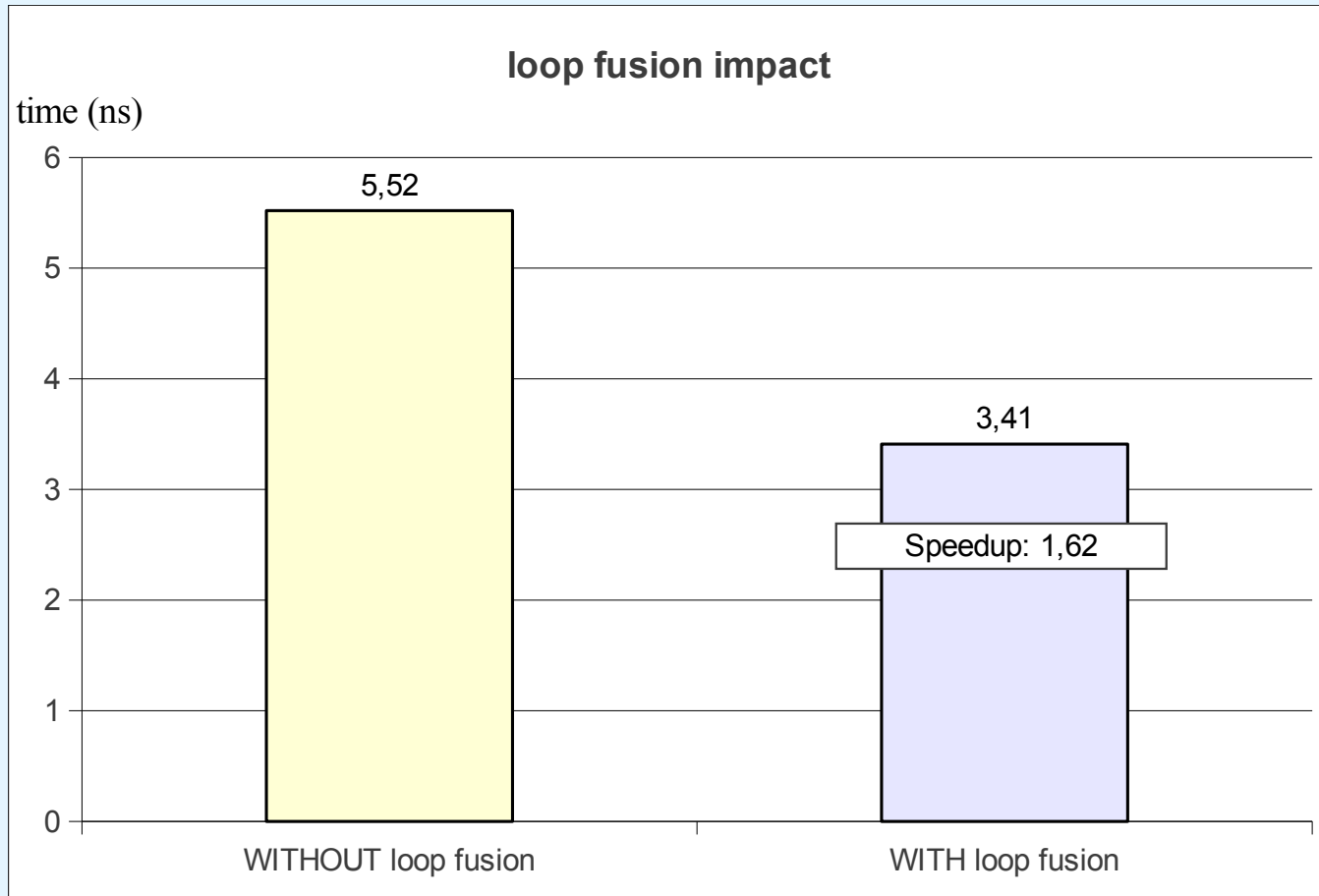
- Técnicas de Otimização de Código

- **loop fusion:** combina ciclos que usam os mesmos dados, melhorando o desempenho da *cache*

```
// SEM loop fusion
for (i=0; i<3000; i++)
    C[i] = A[i] + B[i];
for (i=0; i<3000; i++)
    D[i] = E[i] + C[i];
```

```
// COM loop fusion
for (i=0; i<3000; i++){
    C[i] = A[i] + B[i];
    D[i] = E[i] + C[i];
}
```

7.4 Otimização do desempenho da CPU - Técnicas de Otimização de Código

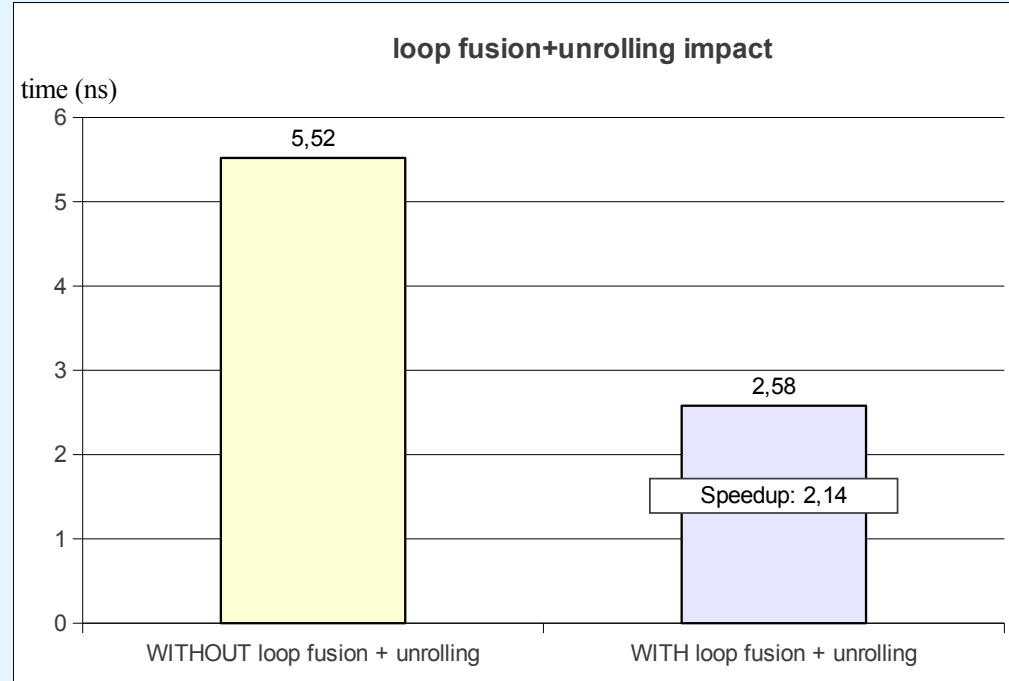


7.4 Otimização do desempenho da CPU

- Técnicas de Otimização de Código

- *loop fusion + unrolling*

```
// COM loop fusion+unrolling
for (i=0; i<3000; i+=3){
    C[i] = A[i] + B[i];
    C[i+1] = A[i+1] + B[i+1];
    C[i+2] = A[i+2] + B[i+2];
    D[i] = E[i] + C[i];
    D[i+1] = E[i+1] + C[i+1];
    D[i+2] = E[i+2] + C[i+2];
}
```



gcc options -O0 -lm, Core i7-8865U 4.8GHz 14 nm (2018)

7.4 Otimização do desempenho da CPU

- Técnicas de Otimização de Código

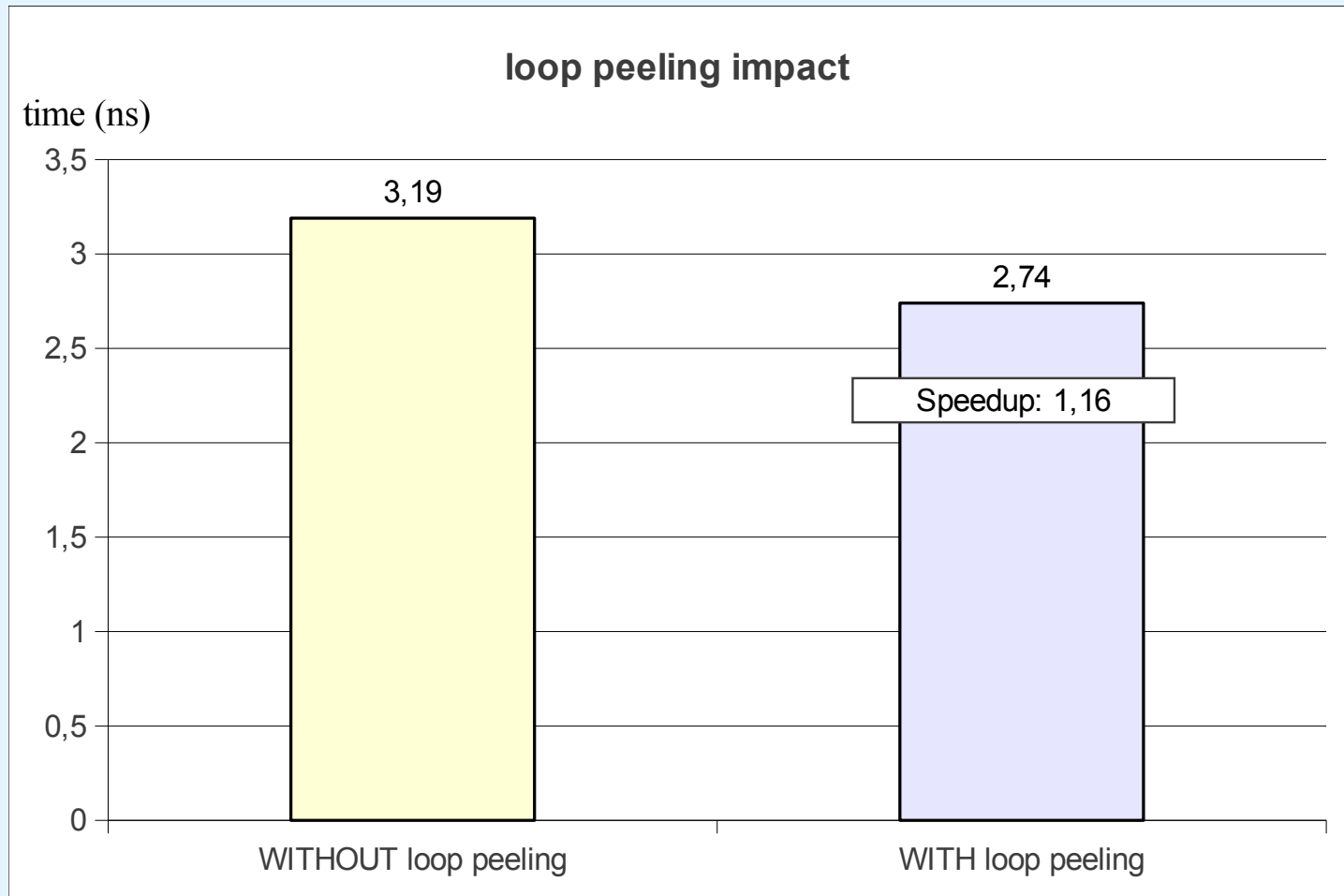
- **loop fission**: divide ciclos extensos em ciclos mais pequenos para reduzir as dependências de dados
 - **loop peeling**: remove o início e o fim de cada ciclo

```
// SEM loop peeling
for (i=0; i<3000; i++)
    if (i == 0)
        A[i] = 0;
    else if (i == 2999)
        A[i] = 2999;
    else
        A[i] += 8;
}
```

```
// COM loop peeling
A[0] = 0;
for (i=1; i<2999; i++)
    A[i] += 8;
A[2999] = 2999;
```

7.4 Otimização do desempenho da CPU

- Técnicas de Otimização de Código



7.4 Otimização do desempenho da CPU

- Técnicas de Otimização de Código

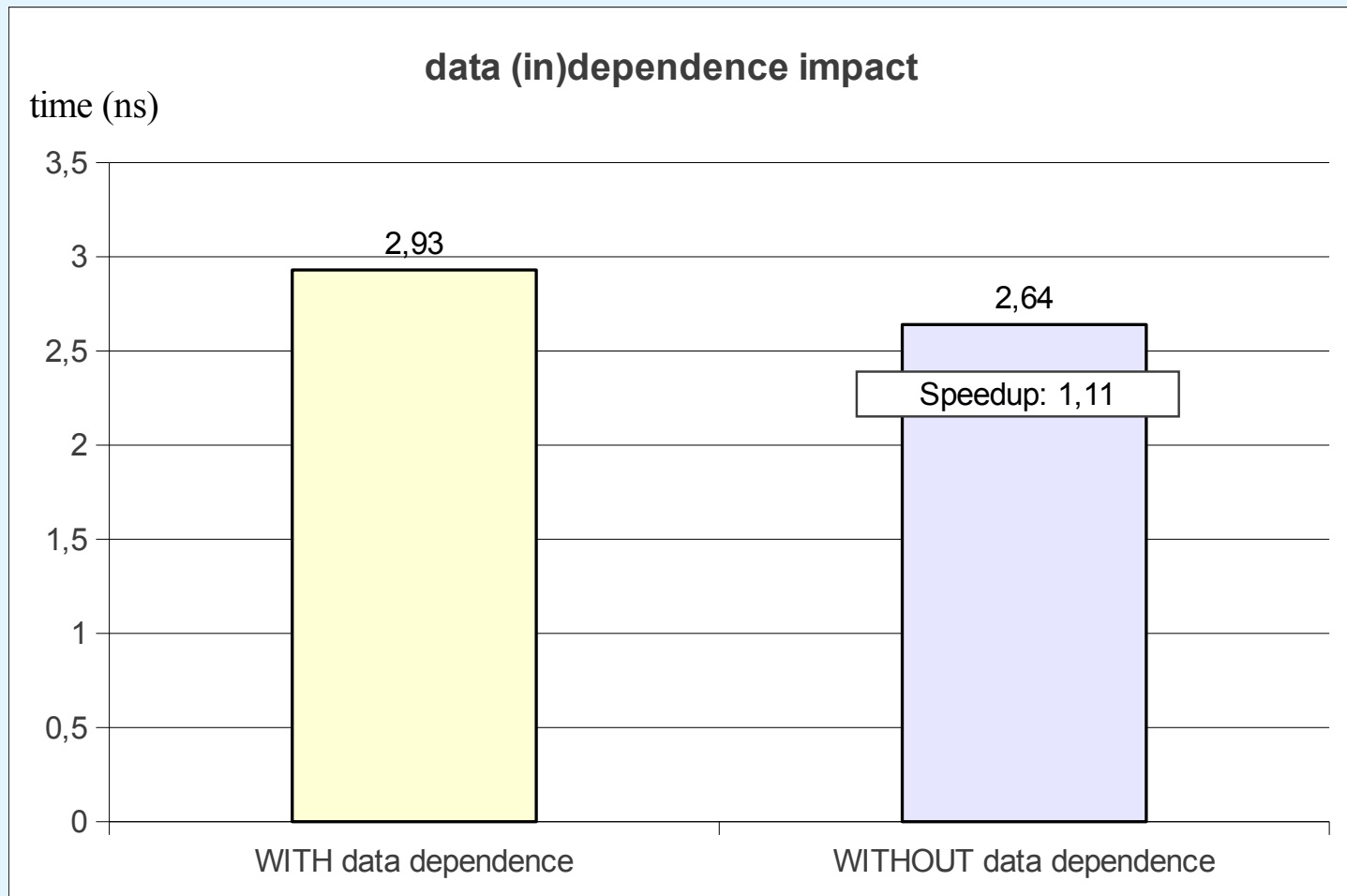
- ***data (in)dependence***: eliminar dependências de dados

```
// COM dependências
for (i=0; i<3000; i++){
    a1 += a2;
    a3 = a1;
    a4 = a5;
}
```

```
// SEM dependências
for (i=0; i<3000; i++){
    a1 += a2;
    a4 = a5;
    a3 = a1;
}
```


7.4 Otimização do desempenho da CPU

- Técnicas de Otimização de Código



7.4 Otimização do desempenho da CPU

- Técnicas de Otimização de Código

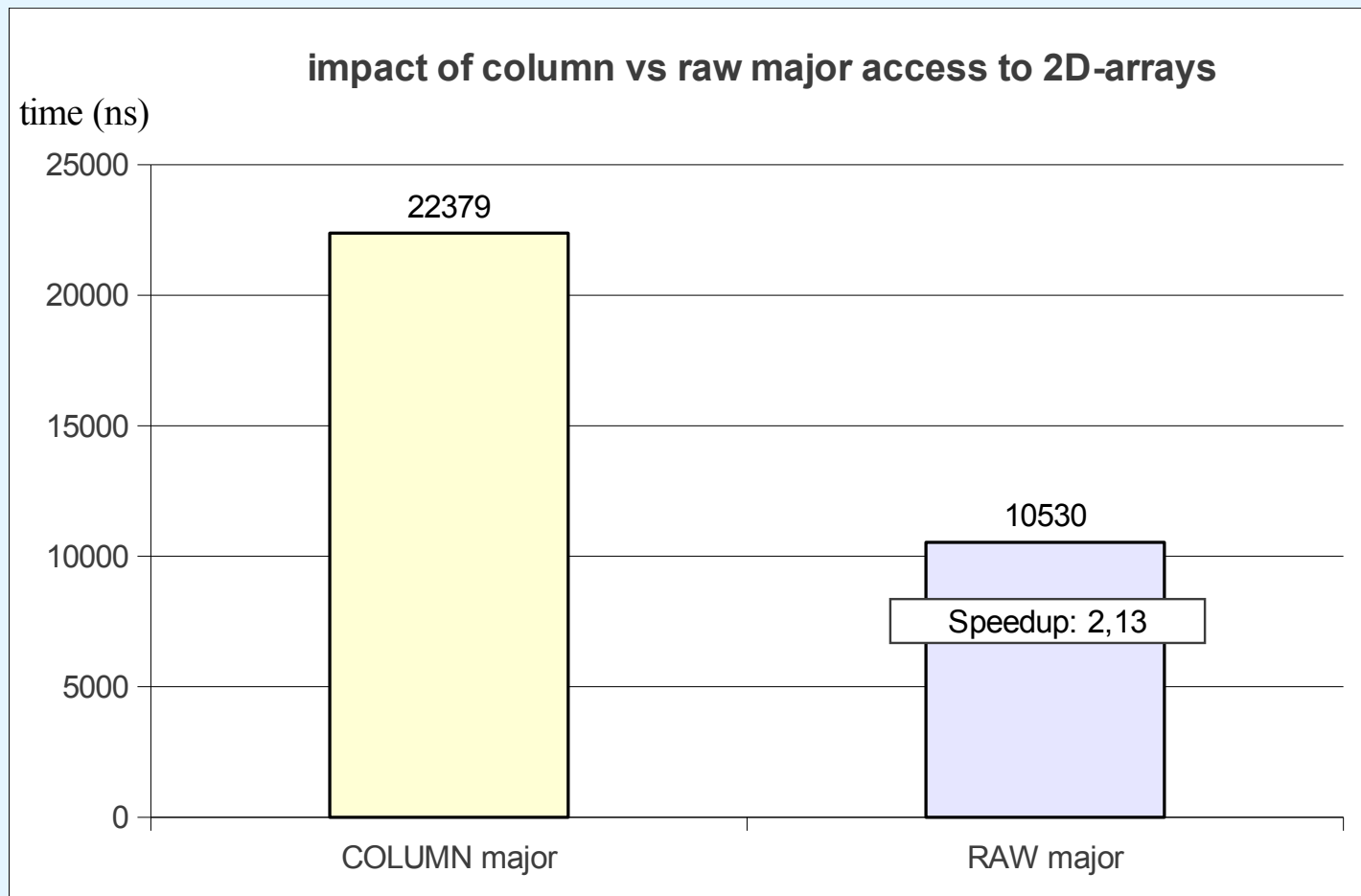
- ***loop interchange***: envolve a re-organização de ciclos para acesso mais eficiente à memória;
- exemplo: acesso a um array bi-dimensional por linha (RAW-major) vs por coluna (COLUMN-major)

```
// acesso COLUMN-major  
for (i=0; i<3000; i++)  
    for (j=0; j<3000; j++)  
        X[j][i]=0;
```

```
// acesso RAW-major  
for (i=0; i<3000; i++)  
    for (j=0; j<3000; j++)  
        X[i][j]=0;
```

7.4 Otimização do desempenho da CPU

- Técnicas de Otimização de Código



7.4 Otimização do desempenho da CPU

- Técnicas de Otimização de Código

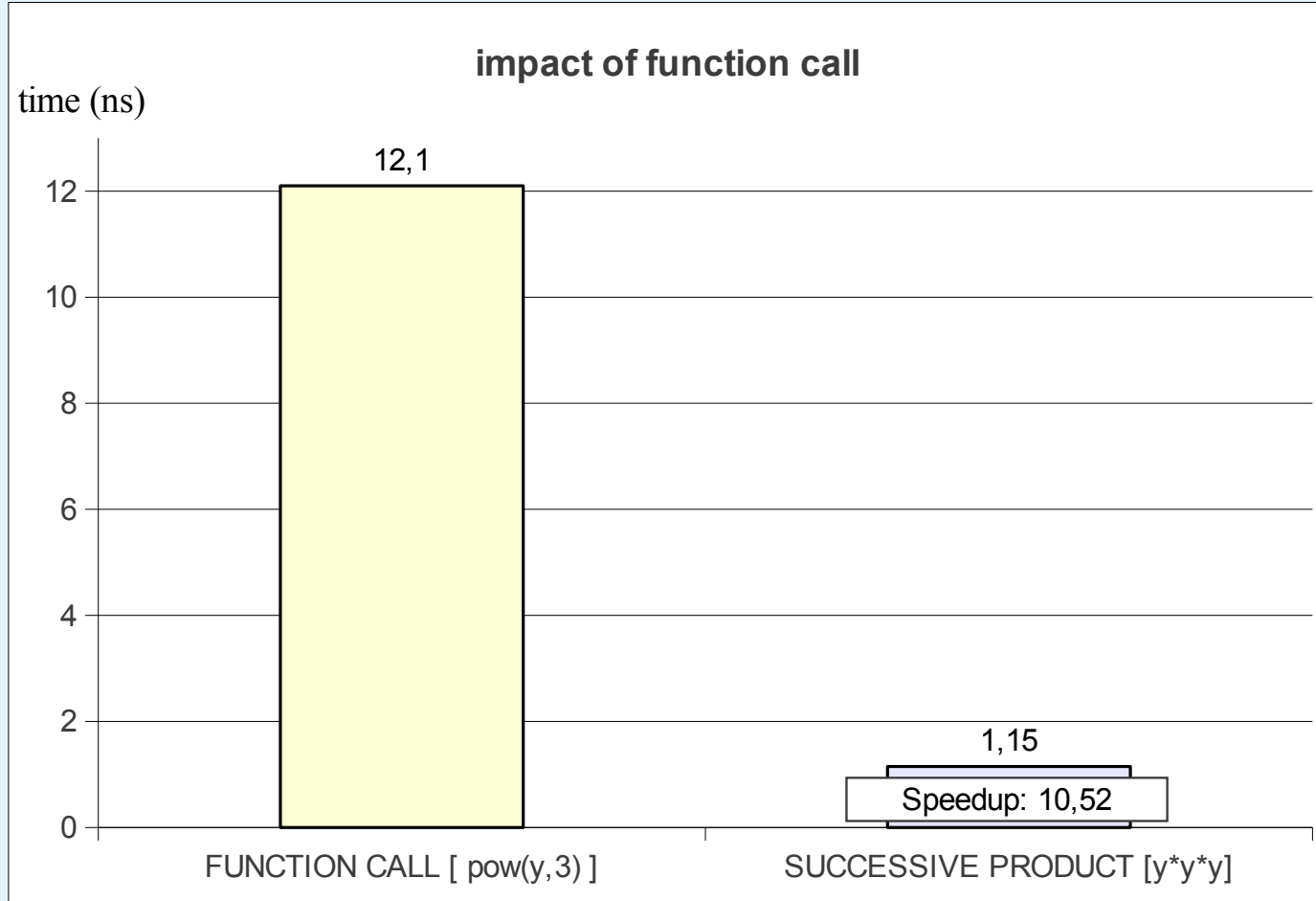
- ***function call avoidance***: evitar invocar rotinas para tarefas que podem ser realizadas localmente, e com isso evitar a sobrecarga da invocação e retorno

```
// função pow para calcular i^3  
for (i=0; i<3000; i++)  
    y += pow(i, 3);
```

```
// produto sucessivo para calcular i^3  
for (i=0; i<3000; i++)  
    y += i * i * i;
```

7.4 Otimização do desempenho da CPU

- Técnicas de Otimização de Código



checkpoint:
atividade
TOC1

7.4 Otimização do desempenho da CPU

- Suporte dos Compiladores

- os **compiladores** suportam **opções** que têm influência sobre espaço e tempo, em compilação e execução
- exemplo: `gcc -Ooptimization_level -funroll-loops prog.c`
 - **-O0** (opção por omissão): sem otimizações; a melhor opção quando se pretende fazer depuração (debug)
 - **-O1**: otimizações com tempo de compilação moderado
 - **-O2**: todas as otimizações que maximizem a velocidade sem aumentar o volume do código; ideal p/ distribuições
 - **-O3**: + (?) desempenho c/ + código (*function inlining*, etc.)
 - **-funroll-loops**: + (?) desempenho, c/ + código

7.4 Otimização do desempenho da CPU

- Suporte dos Compiladores

```
#include <stdio.h>

double powern (double d, unsigned n)
{
    double x = 1.0;
    unsigned j;

    for (j = 1; j <= n; j++)
        x *= d;

    return x;
}

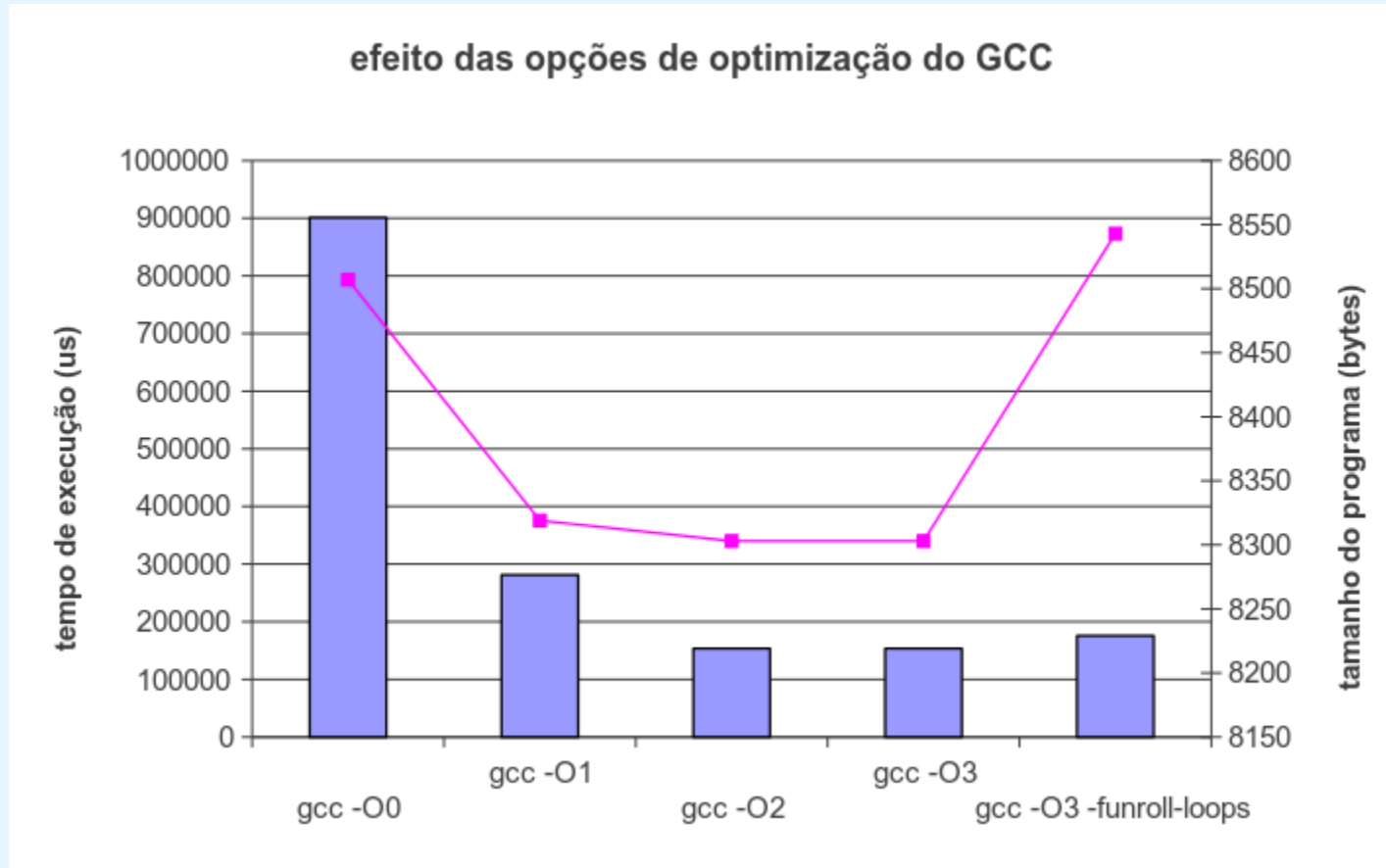
int main (void)
{
    double sum = 0.0;
    unsigned i;

    for (i = 1; i <= 100000000; i++)
    {
        sum += powern (i, i % 5);
    }

    printf ("sum = %g\n", sum);
    return 0;
}
```


7.4 Otimização do desempenho da CPU

- Suporte dos Compiladores



Core i7-2600 3.8GHz 32nm (2011)

checkpoint:
atividade
SC1

7.5 Lei de Amdahl



- o desempenho global de um sistema é o resultado da interação de todos os seus componentes
- o desempenho de um sistema será melhorado de forma mais significativa se for incrementado o desempenho do seu componente mais utilizado
- esta ideia é quantificada pela **lei de Amdahl**:

$$S = \frac{1}{(1-f) + \frac{f}{k}}$$

S : aceleração (*speedup*) global

f : fração de trabalho que cabe ao componente substituído

k : aceleração (*speedup*) específica do componente substituído

7.5 Lei de Amdahl



- a **lei de Amdahl** permite estimar a mudança no desempenho global do sistema, pela intervenção (upgrade/downgrade) em componentes específicos
 - permite ainda realizar **análises custo-benefício**
- **exemplo:**
 - num dado sistema, pode-se melhorar em 50% a CPU com um custo de €10000, ou melhorar os Discos em 150% com um custo de €7000
 - os programas passam 70% do tempo na CPU e 30% do tempo a ser servidos (R/W) pelo Disco
 - qual o upgrade que oferecerá o maior aumento de desempenho ? e qual será mais compensador ?

7.5 Lei de Amdahl



- a opção pela CPU resulta num *speedup* de 130%:

$$\begin{array}{l} f = 0.70, \\ k = 1.5 \end{array} \quad S = \frac{1}{(1 - 0.7) + 0.7/1.5}$$

- a opção pelos Discos produz um *speedup* de 122%:

$$\begin{array}{l} f = 0.30, \\ k = 2.5 \end{array} \quad S = \frac{1}{(1 - 0.3) + 0.3/2.5}$$

- **análise custo/benefício**: cada 1% de *speedup* global adicional à base de CPU custa €10000 / 30% = €333, mas à base dos Discos custa €7000 / 22% = €318
 - com custos semelhantes, outros critérios serão relevantes (e.g., fim de vida dos Discos atuais ou do seu espaço)

7.5 Lei de Amdahl

checkpoint:
exercícios LA1 a LA4

- resumindo: a Lei de Amdahl indica que se deve tornar mais rápido o componente mais utilizado
- assim:
 - se uma dada aplicação depender fortemente da CPU (*CPU bound*) deve-se tornar a CPU mais rápida
 - um sistema que usa muito a RAM (*memory bound*) beneficiará de melhorias ao nível da gestão da memória
 - o desempenho de um sistema muito dependente da E/S (*I/O bound*) melhorará c/ a atualização do sistema E/S

atenção: ao melhorar um dado componente, pode-se colocar a descoberto um problema de desempenho noutro componente !

Conclusões

- a avaliação do desempenho de um computador baseia-se em medições de tendências dominantes, incluindo média aritmética, aritmética pesada, geométrica e harmónica
- existem várias suites de *benchmarking* que permitem avaliar o desempenho de forma objetiva; as mais conceituadas são os *benchmarks* SPEC e TPC
- o desempenho da CPU depende de muitos fatores
- a otimização de código inclui a reestruturação de ciclos e o desenho de algoritmos mais eficientes
- a lei de Amdahl quantifica a influência de up/downgrades nos componentes de Hardware do sistema

Referências



- [0] livro de apoio (ECOIA), Capítulos 7 e 11
- [1] “On Average, You’re Using the Wrong Average: Geometric & Harmonic Means in Data Analysis”,
<https://towardsdatascience.com/on-average-youre-using-the-wrong-average-geometric-harmonic-means-in-data-analysis-2a703e21ea0>
- [2] “SPEC Benchmarks”, <https://www.spec.org/benchmarks.html>
- [3] “TPC-Homepage”, <https://www.tpc.org/>
- [4] “GCC Options That Control Optimization”,
<https://gcc.gnu.org/onlinedocs/gcc/Optimize-Options.html>
- [5] “8 Ways to Measure Execution Time in C/C++”,
<https://levelup.gitconnected.com/8-ways-to-measure-execution-time-in-c-c-48634458d0f9>